

porting_plandex

Plandex -> HelixCode Feature Porting Plan

Source: Plandex (github.com/plandex-ai/plandex) — Go-based AI coding agent, 15K+ stars

Target: HelixCode (github.com/HelixDevelopment/HelixCode) — Distributed AI development platform
Porting Strategy: Deep integration into HelixCode's internal/ packages, respecting existing editor, llm, context, session, workflow, memory, mcp, and agent boundaries.

Feature 1: Cumulative Diff Review Sandbox

Source Location (in Plandex)

- app/server/diff/diff.go — Git-based diff generation, hunk parsing, GetDiffReplacements()
- app/server/types/active_plan.go — ActivePlan with BuildQueuesByPath, BuiltFiles, IsBuildingByPath
- app/server/types/active_plan_pending_builds.go — Pending build management
- CLI: plandex diff, plandex apply, plandex reject — Sandbox review commands

Target Location (in HelixCode)

- **NEW:** internal/editor/diff_sandbox.go — Core sandbox engine
- **NEW:** internal/editor/diff_sandbox_test.go — Sandbox tests
- **MODIFY:** internal/editor/editor.go — Integrate sandbox into CodeEditor
- **MODIFY:** internal/session/session.go — Add SandboxState to Session
- **MODIFY:** cmd/cli/ — Add review, apply, reject CLI commands

Exact Code Changes

NEW FILE: internal/editor/diff_sandbox.go

```
package editor
```

```
import (  
    "bufio"  
    "fmt"  
    "os"  
    "os/exec"  
    "path/filepath"  
    "strconv"  
    "strings"  
    "sync"
```

```

    "time"

    "github.com/google/uuid"
)

// DiffSandbox accumulates AI-generated changes in isolation from
// the working tree.
// Inspired by Plandex's cumulative diff review sandbox.
type DiffSandbox struct {
    mu      sync.RWMutex
    ID      string
    PlanID  string
    Branch  string
    // filePath -> pending edit
    PendingEdits map[string]*PendingEdit
    // Applied edits (for history/rollback)
    AppliedEdits map[string]*AppliedEdit
    CreatedAt    time.Time
    UpdatedAt    time.Time
}

// PendingEdit represents a single file's proposed changes,
// decomposed into hunks.
type PendingEdit struct {
    FilePath      string
    OriginalHash  string          // SHA256 of original file
    content
    OriginalText  string
    ProposedText  string
    Hunks         []*DiffHunk    // Atomic review units
    Status        EditStatus
    CreatedAt     time.Time
}

// DiffHunk is an atomic unit of review (accept/reject).
type DiffHunk struct {
    ID            string
    OldStart      int
    OldLines      int
    NewStart      int
    NewLines      int
    OldText       string
    NewText       string
    Status        HunkStatus
}

type HunkStatus int
const (

```

```

    HunkStatusPending HunkStatus = iota
    HunkStatusAccepted
    HunkStatusRejected
)

type EditStatus int
const (
    EditStatusPending EditStatus = iota
    EditStatusPartial
    EditStatusAccepted
    EditStatusRejected
)

// AppliedEdit tracks what was written to disk (for rewind).
type AppliedEdit struct {
    FilePath      string
    PreviousText  string
    AppliedText   string
    AppliedAt     time.Time
    HunkIDs       []string
}

// NewDiffSandbox creates a sandbox for a plan.
func NewDiffSandbox(planID, branch string) *DiffSandbox {
    return &DiffSandbox{
        ID:          uuid.New().String(),
        PlanID:       planID,
        Branch:       branch,
        PendingEdits: make(map[string]*PendingEdit),
        AppliedEdits: make(map[string]*AppliedEdit),
        CreatedAt:    time.Now(),
        UpdatedAt:    time.Now(),
    }
}

// StageEdit stages a full-file edit into the sandbox, splitting
// into hunks.
func (ds *DiffSandbox) StageEdit(filePath, original, proposed
    string) error {
    ds.mu.Lock()
    defer ds.mu.Unlock()

    hunks, err := computeHunks(original, proposed)
    if err != nil {
        return fmt.Errorf("computeHunks: %w", err)
    }

    ds.PendingEdits[filePath] = &PendingEdit{

```

```

        FilePath:    filePath,
        OriginalText: original,
        ProposedText: proposed,
        Hunks:       hunks,
        Status:       EditStatusPending,
        CreatedAt:    time.Now(),
    }
    ds.UpdatedAt = time.Now()
    return nil
}

// computeHunks uses git diff --no-index to generate atomic hunks.
func computeHunks(original, proposed string) ([]*DiffHunk, error) {
    tempDir, err := os.MkdirTemp("", "helix-sandbox-*")
    if err != nil {
        return nil, err
    }
    defer os.RemoveAll(tempDir)

    origPath := filepath.Join(tempDir, "original")
    propPath := filepath.Join(tempDir, "proposed")
    os.WriteFile(origPath, []byte(original), 0644)
    os.WriteFile(propPath, []byte(proposed), 0644)

    cmd := exec.Command("git", "-C", tempDir, "diff", "--no-color", "--no-index", "-U3", "original", "proposed")
    out, err := cmd.CombinedOutput()
    if err != nil {
        if exitErr, ok := err.(*exec.ExitError); !ok || exitErr.ExitCode() != 1 {
            return nil, fmt.Errorf("git diff failed: %v", err)
        }
    }

    return parseGitDiff(string(out)), nil
}

// parseGitDiff parses unified diff into hunks.
func parseGitDiff(diff string) []*DiffHunk {
    var hunks []*DiffHunk
    scanner := bufio.NewScanner(strings.NewReader(diff))
    var current *DiffHunk
    var oldLines, newLines []string

    for scanner.Scan() {
        line := scanner.Text()

```

```

if strings.HasPrefix(line, "@@") {
    if current != nil {
        current.OldText = strings.Join(oldLines, "\n")
        current.NewText = strings.Join(newLines, "\n")
        hunks = append(hunks, current)
    }
    parts := strings.Split(line, " ")
    oldInfo :=
strings.Split(strings.TrimPrefix(parts[1], "-"), ",")
    newInfo :=
strings.Split(strings.TrimPrefix(parts[2], "+"), ",")
    oldStart, _ := strconv.Atoi(oldInfo[0])
    oldLen := 0
    if len(oldInfo) > 1 {
        oldLen, _ = strconv.Atoi(oldInfo[1])
    }
    newStart, _ := strconv.Atoi(newInfo[0])
    newLen := 0
    if len(newInfo) > 1 {
        newLen, _ = strconv.Atoi(newInfo[1])
    }
    current = &DiffHunk{
        ID:      uuid.New().String(),
        OldStart: oldStart,
        OldLines: oldLen,
        NewStart: newStart,
        NewLines: newLen,
        Status:   HunkStatusPending,
    }
    oldLines, newLines = nil, nil
} else if current != nil {
    switch {
    case strings.HasPrefix(line, "-") && !
strings.HasPrefix(line, "---"):
        oldLines = append(oldLines,
strings.TrimPrefix(line, "-"))
    case strings.HasPrefix(line, "+") && !
strings.HasPrefix(line, "+++"):
        newLines = append(newLines,
strings.TrimPrefix(line, "+"))
    case strings.HasPrefix(line, " "):
        oldLines = append(oldLines,
strings.TrimPrefix(line, " "))
        newLines = append(newLines,
strings.TrimPrefix(line, " "))
    }
}
}

```

```

    if current != nil {
        current.OldText = strings.Join(oldLines, "\n")
        current.NewText = strings.Join(newLines, "\n")
        hunks = append(hunks, current)
    }
    return hunks
}

// AcceptHunk marks a hunk as accepted.
func (ds *DiffSandbox) AcceptHunk(filePath, hunkID string) error
{
    ds.mu.Lock()
    defer ds.mu.Unlock()

    edit, ok := ds.PendingEdits[filePath]
    if !ok {
        return fmt.Errorf("no pending edit for %s", filePath)
    }
    for _, h := range edit.Hunks {
        if h.ID == hunkID {
            h.Status = HunkStatusAccepted
        }
    }
    ds.recalcEditStatus(edit)
    return nil
}

// RejectHunk marks a hunk as rejected.
func (ds *DiffSandbox) RejectHunk(filePath, hunkID string) error
{
    ds.mu.Lock()
    defer ds.mu.Unlock()

    edit, ok := ds.PendingEdits[filePath]
    if !ok {
        return fmt.Errorf("no pending edit for %s", filePath)
    }
    for _, h := range edit.Hunks {
        if h.ID == hunkID {
            h.Status = HunkStatusRejected
        }
    }
    ds.recalcEditStatus(edit)
    return nil
}

func (ds *DiffSandbox) recalcEditStatus(edit *PendingEdit) {
    hasAccepted, hasPending := false, false

```

```

for _, h := range edit.Hunks {
    switch h.Status {
    case HunkStatusAccepted:
        hasAccepted = true
    case HunkStatusPending:
        hasPending = true
    }
}
switch {
case hasPending && hasAccepted:
    edit.Status = EditStatusPartial
case hasAccepted && !hasPending:
    edit.Status = EditStatusAccepted
case !hasAccepted && !hasPending:
    edit.Status = EditStatusRejected
default:
    edit.Status = EditStatusPending
}
}

// BuildFinalText reconstructs proposed text after applying
// accepted/rejected hunks.
func (ds *DiffSandbox) BuildFinalText(filePath string) (string,
    error) {
    ds.mu.RLock()
    defer ds.mu.RUnlock()

    edit, ok := ds.PendingEdits[filePath]
    if !ok {
        return "", fmt.Errorf("no pending edit for %s", filePath)
    }

    // Start from original, apply hunks in reverse order to
    // preserve line numbers
    lines := strings.Split(edit.OriginalText, "\n")
    for i := len(edit.Hunks) - 1; i >= 0; i-- {
        h := edit.Hunks[i]
        switch h.Status {
        case HunkStatusAccepted:
            // Replace oldText with newText
            oldL := strings.Split(h.OldText, "\n")
            newL := strings.Split(h.NewText, "\n")
            start := h.OldStart - 1 // 1-indexed to 0-indexed
            if start < 0 {
                start = 0
            }
            end := start + len(oldL)
            if end > len(lines) {

```

```

        end = len(lines)
    }
    before := lines[:start]
    after := lines[end:]
    lines = append(before, append(newL, after...)...)
case HunkStatusRejected:
    // Keep original (no-op)
case HunkStatusPending:
    return "", fmt.Errorf("hunk %s still pending", h.ID)
}
}
return strings.Join(lines, "\n"), nil
}

// ApplyAll writes all accepted edits to disk atomically.
func (ds *DiffSandbox) ApplyAll(baseDir string)
    (map[string]string, error) {
    ds.mu.Lock()
    defer ds.mu.Unlock()

    results := make(map[string]string)
    for path, edit := range ds.PendingEdits {
        if edit.Status != EditStatusAccepted && edit.Status !=
            EditStatusPartial {
            continue
        }
        finalText, err := ds.buildFinalTextUnlocked(edit)
        if err != nil {
            return nil, fmt.Errorf("build final for %s: %w",
                path, err)
        }
        fullPath := filepath.Join(baseDir, path)
        // Store previous for rollback
        prev, _ := os.ReadFile(fullPath)
        ds.AppliedEdits[path] = &AppliedEdit{
            FilePath:    path,
            PreviousText: string(prev),
            AppliedText:   finalText,
            AppliedAt:     time.Now(),
        }
        if err := os.WriteFile(fullPath, []byte(finalText),
            0644); err != nil {
            return nil, fmt.Errorf("write %s: %w", path, err)
        }
        results[path] = finalText
    }
    ds.UpdatedAt = time.Now()
    return results, nil
}

```



```

}

func (ds *DiffSandbox) buildFinalTextUnlocked(edit *PendingEdit)
    (string, error) {
    lines := strings.Split(edit.OriginalText, "\n")
    for i := len(edit.Hunks) - 1; i >= 0; i-- {
        h := edit.Hunks[i]
        if h.Status == HunkStatusAccepted {
            oldL := strings.Split(h.OldText, "\n")
            newL := strings.Split(h.NewText, "\n")
            start := h.OldStart - 1
            if start < 0 {
                start = 0
            }
            end := start + len(oldL)
            if end > len(lines) {
                end = len(lines)
            }
            before := lines[:start]
            after := lines[end:]
            lines = append(before, append(newL, after...)...)
        }
    }
    return strings.Join(lines, "\n"), nil
}

// RevertAll rolls back applied edits to their previous state.
func (ds *DiffSandbox) RevertAll(baseDir string) error {
    ds.mu.Lock()
    defer ds.mu.Unlock()

    for path, applied := range ds.AppliedEdits {
        fullPath := filepath.Join(baseDir, path)
        if err := os.WriteFile(fullPath,
            []byte(applied.PreviousText), 0644); err != nil {
            return fmt.Errorf("revert %s: %w", path, err)
        }
    }
    return nil
}

// GetStats returns review statistics.
func (ds *DiffSandbox) GetStats() (total, accepted, rejected,
    pending int) {
    ds.mu.RLock()
    defer ds.mu.RUnlock()

    for _, edit := range ds.PendingEdits {

```

```

        for _, h := range edit.Hunks {
            total++
            switch h.Status {
            case HunkStatusAccepted:
                accepted++
            case HunkStatusRejected:
                rejected++
            case HunkStatusPending:
                pending++
            }
        }
    }
    return
}

```

MODIFY: internal/editor/editor.go

Add to CodeEditor:

```

// Sandbox is the cumulative diff review sandbox for this editor
// session.
Sandbox *DiffSandbox

// NewCodeEditorWithSandbox creates an editor with sandbox
// support.
func NewCodeEditorWithSandbox(format EditFormat, planID, branch
    string) (*CodeEditor, error) {
    ce, err := NewCodeEditor(format)
    if err != nil {
        return nil, err
    }
    ce.Sandbox = NewDiffSandbox(planID, branch)
    return ce, nil
}

// StageEdit stages an AI-generated edit in the sandbox instead
// of applying directly.
func (ce *CodeEditor) StageEdit(edit Edit) (*PendingEdit, error)
{
    ce.mu.Lock()
    defer ce.mu.Unlock()

    if ce.Sandbox == nil {
        return nil, fmt.Errorf("sandbox not initialized")
    }

    var original string
    if data, err := os.ReadFile(edit.FilePath); err == nil {
        original = string(data)
    }
}

```

```

    }

    var proposed string
    switch edit.Format {
    case EditFormatWhole:
        proposed = edit.Content.(string)
    case EditFormatDiff:
        // Apply diff to original to get proposed
        proposed = applyDiff(original, edit.Content.(string))
    default:
        return nil, fmt.Errorf("sandbox only supports whole/diff
            formats")
    }

    if err := ce.Sandbox.StageEdit(edit.FilePath, original,
        proposed); err != nil {
        return nil, err
    }
    return ce.Sandbox.PendingEdits[edit.FilePath], nil
}

```

MODIFY: internal/session/session.go

Add to Session:

```

// SandboxID references the active diff sandbox for this session.
SandboxID string `json:"sandbox_id,omitempty"`

```

Anti-Bluff Test

```

# 1. Create a test Go project
cd /tmp/test_sandbox
echo 'package main\n\nfunc Hello() string {\n    return
    "old"\n}' > main.go

# 2. Start a HelixCode session with sandbox
helix code --sandbox --plan test-plan-1

# 3. Stage an AI edit
# POST /api/v1/sessions/{id}/sandbox/stage
# Body: {"file_path": "main.go", "format": "whole", "content":
    "package main\n\nfunc Hello() string {\n    return
    \"new\"\n}"}

# 4. Verify file unchanged on disk
grep -q 'return "old"' main.go && echo "PASS: original preserved"

# 5. Review hunks
helix sandbox diff main.go # Shows hunk-level diff

```

6. Accept the hunk

```
helix sandbox accept --hunk {hunk-id} main.go
```

7. Apply all accepted

```
helix sandbox apply
```

8. Verify file changed

```
grep -q 'return "new"' main.go && echo "PASS: edit applied"
```

9. Revert

```
helix sandbox revert
```

10. Verify rollback

```
grep -q 'return "old"' main.go && echo "PASS: rollback works"
```

Integration Verification

☐

go test ./internal/editor/... passes with sandbox tests

☐

CLI commands review, apply, reject exist in cmd/cli/

☐

Session API endpoints for sandbox CRUD

Feature 2: 2M Token Context + 20M Token Indexing

Source Location (in Plandex)

- app/server/syntax/file_map/ — Tree-sitter based project maps
- app/server/syntax/parsers.go — Tree-sitter parsers for 30+ languages
- app/server/syntax/structured_edits_tree_sitter.go — Syntax-aware edits
- app/server/model/tokens.go — Token counting
- app/server/db/context.go — Context loading/storing

Target Location (in HelixCode)

- **NEW:** internal/context/massive_context.go — Massive context orchestrator
- **NEW:** internal/context/indexer.go — Tree-sitter indexing engine
- **NEW:** internal/context/chunker.go — Content chunking strategy
- **NEW:** internal/context/retriever.go — Embedding-based retrieval
- **MODIFY:** internal/context/context_manager.go — Integrate massive context
- **MODIFY:** internal/discovery/ — Project discovery integration

Exact Code Changes

NEW FILE: internal/context/massive_context.go

```
package context

import (
    "context"
    "fmt"
    "os"
    "path/filepath"
    "strings"
    "sync"
    "time"

    "dev.helix.code/internal/llm"
)

// MaxDirectContextTokens is the max tokens sent directly to the
// LLM (2M for Plandex default pack).
const MaxDirectContextTokens = 2_000_000

// MaxIndexedTokens is the max tokens that can be indexed for
// retrieval (20M+).
const MaxIndexedTokens = 20_000_000

// MassiveContext manages the 2M/20M token window strategy.
type MassiveContext struct {
    mu sync.RWMutex

    // Project root
    Root string

    // indexer builds and maintains the tree-sitter project map
    Indexer *ProjectIndexer

    // chunker splits large files into ~100k token chunks
    Chunker *ContentChunker

    // retriever does semantic search over indexed chunks
    Retriever *EmbeddingRetriever

    // activeContext is what's currently loaded into the LLM
    // context window
    activeContext map[string]*ContextFile

    // totalTokens tracks current context token count
    totalTokens int
}
```

```

// ContextFile represents a loaded file with its chunk.
type ContextFile struct {
    Path          string
    Content       string
    Tokens        int
    ChunkIndex    int // Which chunk of a large file (0 = first)
    TotalChunks   int
    LoadedAt      time.Time
}

// NewMassiveContext initializes the context system.
func NewMassiveContext(root string, embedder llm.Embedder)
    (*MassiveContext, error) {
    indexer, err := NewProjectIndexer(root)
    if err != nil {
        return nil, fmt.Errorf("indexer: %w", err)
    }
    chunker := NewContentChunker()
    retriever, err := NewEmbeddingRetriever(embedder)
    if err != nil {
        return nil, fmt.Errorf("retriever: %w", err)
    }

    mc := &MassiveContext{
        Root:          root,
        Indexer:        indexer,
        Chunker:        chunker,
        Retriever:      retriever,
        activeContext: make(map[string]*ContextFile),
    }

    // Initial indexing
    if err := mc.BuildIndex(); err != nil {
        return nil, fmt.Errorf("buildIndex: %w", err)
    }
    return mc, nil
}

// BuildIndex scans the project and creates the tree-sitter map +
// vector DB.
func (mc *MassiveContext) BuildIndex() error {
    mc.mu.Lock()
    defer mc.mu.Unlock()

    // Step 1: Generate tree-sitter project map
    if err := mc.Indexer.BuildMap(); err != nil {
        return fmt.Errorf("build map: %w", err)
    }

```

```

}

// Step 2: Chunk all files > 100k tokens
files := mc.Indexer.GetAllFiles()
var chunks []*ContentChunk
for _, f := range files {
    data, err := os.ReadFile(filepath.Join(mc.Root, f.Path))
    if err != nil {
        continue
    }
    fileChunks := mc.Chunker.Chunk(string(data), f.Path)
    chunks = append(chunks, fileChunks...)
}

// Step 3: Embed and index chunks
ctx, cancel := context.WithTimeout(context.Background(),
    30*time.Minute)
defer cancel()
return mc.Retriever.Index(ctx, chunks)
}

// LoadContextForQuery retrieves the most relevant context for a
// user query.
func (mc *MassiveContext) LoadContextForQuery(ctx
    context.Context, query string, maxTokens int)
    ([]*ContextFile, error) {
    mc.mu.Lock()
    defer mc.mu.Unlock()

    if maxTokens > MaxDirectContextTokens {
        maxTokens = MaxDirectContextTokens
    }

    // Step 1: Semantic search over indexed chunks
    results, err := mc.Retriever.Search(ctx, query, 50)
    if err != nil {
        return nil, fmt.Errorf("semantic search: %w", err)
    }

    // Step 2: Follow import chains (structural relevance)
    relevantPaths := mc.followImportChains(results)

    // Step 3: Load files up to maxTokens, prioritizing by
    // relevance score
    var loaded []*ContextFile
    tokensUsed := 0
    for _, path := range relevantPaths {
        if tokensUsed >= maxTokens {

```

```

        break
    }
    cf, err := mc.loadFile(path)
    if err != nil {
        continue
    }
    if tokensUsed+cf.Tokens > maxTokens {
        // Try to load just the most relevant chunk
        chunk := mc.getMostRelevantChunk(path, query)
        if chunk != nil && tokensUsed+chunk.Tokens <=
maxTokens {
            loaded = append(loaded, chunk)
            tokensUsed += chunk.Tokens
        }
        continue
    }
    loaded = append(loaded, cf)
    tokensUsed += cf.Tokens
}

mc.totalTokens = tokensUsed
return loaded, nil
}

// loadFile loads a full file into context.
func (mc *MassiveContext) loadFile(path string) (*ContextFile,
error) {
    if cf, ok := mc.activeContext[path]; ok {
        return cf, nil
    }
    data, err := os.ReadFile(filepath.Join(mc.Root, path))
    if err != nil {
        return nil, err
    }
    content := string(data)
    tokens := llm.EstimateTokens(content) // uses tiktoken or
similar
    cf := &ContextFile{
        Path:      path,
        Content:    content,
        Tokens:     tokens,
        ChunkIndex: 0,
        TotalChunks: 1,
        LoadedAt:   time.Now(),
    }
    mc.activeContext[path] = cf
    return cf, nil
}

```



```

func (mc *MassiveContext) getMostRelevantChunk(path, query
    string) *ContextFile {
    // Returns the best single chunk for a large file based on
    semantic similarity
    chunks := mc.Chunker.GetChunks(path)
    if len(chunks) == 0 {
        return nil
    }
    // Simplified: return first chunk; in production, run
    embedding comparison
    return &ContextFile{
        Path:      path,
        Content:    chunks[0].Text,
        Tokens:     chunks[0].Tokens,
        ChunkIndex: chunks[0].Index,
        LoadedAt:   time.Now(),
    }
}

func (mc *MassiveContext) followImportChains(initial []string)
    []string {
    // Uses the tree-sitter project map to find related files via
    imports/calls
    seen := make(map[string]bool)
    var result []string
    for _, p := range initial {
        if seen[p] {
            continue
        }
        seen[p] = true
        result = append(result, p)
        // Add files that import this file or are imported by it
        related := mc.Indexer.GetRelatedFiles(p)
        for _, r := range related {
            if !seen[r] {
                seen[r] = true
                result = append(result, r)
            }
        }
    }
    return result
}

```

NEW FILE: internal/context/indexer.go

```
package context
```

```

import (
    "fmt"
    "os"
    "path/filepath"
    "strings"

    sitter "github.com/smacker/go-tree-sitter"
    "github.com/smacker/go-tree-sitter/golang"
    "github.com/smacker/go-tree-sitter/javascript"
    "github.com/smacker/go-tree-sitter/python"
    // ... 30+ languages as needed
)

// ProjectIndexer builds a structural map of the codebase using
// tree-sitter.
type ProjectIndexer struct {
    Root      string
    Parser    *sitter.Parser
    Files     map[string]*IndexedFile
}

// IndexedFile holds structural metadata for a single file.
type IndexedFile struct {
    Path        string
    Language    string
    Definitions []SymbolDef // Functions, structs, classes
    Imports     []string      // Import paths
    Calls       []string      // Function calls (for call graph)
    Tokens      int
}

type SymbolDef struct {
    Name      string
    Type      string // "function", "struct", "class", "method"
    LineStart int
    LineEnd   int
}

func NewProjectIndexer(root string) (*ProjectIndexer, error) {
    return &ProjectIndexer{
        Root:    root,
        Parser:  sitter.NewParser(),
        Files:   make(map[string]*IndexedFile),
    }, nil
}

// BuildMap walks the project and parses every source file.
func (pi *ProjectIndexer) BuildMap() error {

```

```

return filepath.Walk(pi.Root, func(path string, info
    os.FileInfo, err error) error {
    if err != nil || info.IsDir() {
        return nil
    }
    rel, _ := filepath.Rel(pi.Root, path)
    lang := detectLanguage(path)
    if lang == "" {
        return nil
    }
    data, err := os.ReadFile(path)
    if err != nil {
        return nil
    }
    idx := pi.parseFile(rel, lang, string(data))
    pi.Files[rel] = idx
    return nil
})
}

func (pi *ProjectIndexer) parseFile(path, lang, content string)
    *IndexedFile {
    // Set parser language
    switch lang {
    case "go":
        pi.Parser.SetLanguage(golang.GetLanguage())
    case "js", "ts":
        pi.Parser.SetLanguage(javascript.GetLanguage())
    case "py":
        pi.Parser.SetLanguage(python.GetLanguage())
    }

    tree := pi.Parser.ParseString(nil, content)
    root := tree.RootNode()

    idx := &IndexedFile{
        Path:      path,
        Language:   lang,
        Tokens:     llm.EstimateTokens(content),
    }

    // Walk AST to extract definitions and imports
    pi.walkAST(root, content, idx)
    return idx
}

func (pi *ProjectIndexer) walkAST(node *sitter.Node, content
    string, idx *IndexedFile) {

```

```

    if node == nil {
        return
    }
    switch node.Type() {
    case "function_declaration", "method_declaration":
        nodeName := node.ChildByFieldName("name")
        if nodeName != nil {
            idx.Definitions = append(idx.Definitions, SymbolDef{
                Name:
                content[nameNode.StartByte():nodeName.EndByte()],
                Type:      "function",
                LineStart: int(node.StartPoint().Row),
                LineEnd:  int(node.EndPoint().Row),
            })
        }
    case "import_declaration", "import_spec":
        idx.Imports = append(idx.Imports,
            content[node.StartByte():node.EndByte()])
    case "call_expression":
        fnNode := node.ChildByFieldName("function")
        if fnNode != nil {
            idx.Calls = append(idx.Calls,
                content[fnNode.StartByte():fnNode.EndByte()])
        }
    }
    for i := 0; i < int(node.ChildCount()); i++ {
        pi.walkAST(node.Child(i), content, idx)
    }
}

func (pi *ProjectIndexer) GetAllFiles() []*IndexedFile {
    var out []*IndexedFile
    for _, f := range pi.Files {
        out = append(out, f)
    }
    return out
}

func (pi *ProjectIndexer) GetRelatedFiles(path string) []string {
    file, ok := pi.Files[path]
    if !ok {
        return nil
    }
    var related []string
    // Find files that import the same packages
    for _, other := range pi.Files {
        if other.Path == path {
            continue
        }
    }
}

```

```

    }
    for _, imp := range file.Imports {
        for _, oImp := range other.Imports {
            if strings.Contains(oImp, imp) ||
strings.Contains(imp, oImp) {
                related = append(related, other.Path)
            }
        }
    }
}
return related
}

```

```

func detectLanguage(path string) string {
    switch filepath.Ext(path) {
    case ".go":
        return "go"
    case ".js":
        return "js"
    case ".ts", ".tsx":
        return "ts"
    case ".py":
        return "py"
    case ".rs":
        return "rs"
    case ".c", ".h":
        return "c"
    case ".cpp", ".hpp":
        return "cpp"
    case ".java":
        return "java"
    default:
        return ""
    }
}

```

NEW FILE: internal/context/chunker.go

```

package context

import (
    "strings"
    "sync"
)

// ChunkSizeTokens is ~100k tokens per file for direct loading
// (Plandex default).
const ChunkSizeTokens = 100_000

```

```

// ContentChunker splits files into token-sized chunks.
type ContentChunker struct {
    mu      sync.RWMutex
    chunks map[string][]*ContentChunk // path -> chunks
}

type ContentChunk struct {
    Index  int
    Text   string
    Tokens int
}

func NewContentChunker() *ContentChunker {
    return &ContentChunker{
        chunks: make(map[string][]*ContentChunk),
    }
}

// Chunk splits content into ~ChunkSizeTokens pieces at natural
// boundaries.
func (c *ContentChunker) Chunk(content, path string)
    []*ContentChunk {
    c.mu.Lock()
    defer c.mu.Unlock()

    if existing, ok := c.chunks[path]; ok {
        return existing
    }

    lines := strings.Split(content, "\n")
    var chunks []*ContentChunk
    var current strings.Builder
    currentTokens := 0

    for _, line := range lines {
        lineTokens := llm.EstimateTokens(line)
        if currentTokens+lineTokens > ChunkSizeTokens &&
            current.Len() > 0 {
            chunks = append(chunks, &ContentChunk{
                Index: len(chunks),
                Text:   current.String(),
                Tokens: currentTokens,
            })
            current.Reset()
            currentTokens = 0
        }
        current.WriteString(line)
    }

```

```

        current.WriteString("\n")
        currentTokens += lineTokens
    }
    if current.Len() > 0 {
        chunks = append(chunks, &ContentChunk{
            Index: len(chunks),
            Text:   current.String(),
            Tokens: currentTokens,
        })
    }

    c.chunks[path] = chunks
    return chunks
}

func (c *ContentChunker) GetChunks(path string) []*ContentChunk {
    c.mu.RLock()
    defer c.mu.RUnlock()
    return c.chunks[path]
}

```

NEW FILE: internal/context/retriever.go

```

package context

import (
    "context"
    "fmt"
    "math"
    "sort"
    "sync"

    "dev.helix.code/internal/llm"
)

// EmbeddingRetriever manages semantic search over code chunks.
type EmbeddingRetriever struct {
    mu          sync.RWMutex
    embedder    llm.Embedder
    index       map[string]*IndexedChunk // chunkID -> chunk
    vectors     map[string][]float32     // chunkID -> embedding
}

type IndexedChunk struct {
    ID       string
    Path     string
    Index    int
    Text     string
}

```

```

    Tokens    int
}

func NewEmbeddingRetriever(embedder llm.Embedder)
    (*EmbeddingRetriever, error) {
    return &EmbeddingRetriever{
        embedder: embedder,
        index:    make(map[string]*IndexedChunk),
        vectors:  make(map[string][]float32),
    }, nil
}

func (er *EmbeddingRetriever) Index(ctx context.Context, chunks
    []*ContentChunk) error {
    er.mu.Lock()
    defer er.mu.Unlock()

    var texts []string
    for _, c := range chunks {
        id := fmt.Sprintf("%s#%d", c.Path, c.Index)
        er.index[id] = &IndexedChunk{
            ID:      id,
            Path:    c.Path,
            Index:   c.Index,
            Text:    c.Text,
            Tokens:  c.Tokens,
        }
        texts = append(texts, c.Text)
    }

    vectors, err := er.embedder.EmbedBatch(ctx, texts)
    if err != nil {
        return fmt.Errorf("embed batch: %w", err)
    }
    for i, v := range vectors {
        id := fmt.Sprintf("%s#%d", chunks[i].Path,
            chunks[i].Index)
        er.vectors[id] = v
    }
    return nil
}

type SearchResult struct {
    Path string
    Score float64
}

```



```

func (er *EmbeddingRetriever) Search(ctx context.Context, query
    string, topK int) ([]string, error) {
    er.mu.RLock()
    defer er.mu.RUnlock()

    queryVec, err := er.embedder.Embed(ctx, query)
    if err != nil {
        return nil, fmt.Errorf("embed query: %w", err)
    }

    var results []SearchResult
    for id, vec := range er.vectors {
        chunk := er.index[id]
        score := cosineSimilarity(queryVec, vec)
        results = append(results, SearchResult{Path: chunk.Path,
            Score: score})
    }

    sort.Slice(results, func(i, j int) bool {
        return results[i].Score > results[j].Score
    })

    seen := make(map[string]bool)
    var out []string
    for _, r := range results {
        if seen[r.Path] {
            continue
        }
        seen[r.Path] = true
        out = append(out, r.Path)
        if len(out) >= topK {
            break
        }
    }
    return out, nil
}

func cosineSimilarity(a, b []float32) float64 {
    var dot, normA, normB float64
    for i := range a {
        dot += float64(a[i]) * float64(b[i])
        normA += float64(a[i]) * float64(a[i])
        normB += float64(b[i]) * float64(b[i])
    }
    if normA == 0 || normB == 0 {
        return 0
    }
}

```

```

    return dot / (math.Sqrt(normA) * math.Sqrt(normB))
}

```

MODIFY: internal/context/context_manager.go

Add to ContextManager:

```

// Massive handles the 2M/20M token context strategy.
Massive *MassiveContext

func (cm *ContextManager) SetMassiveContext(mc *MassiveContext) {
    cm.mu.Lock()
    defer cm.mu.Unlock()
    cm.Massive = mc
}

func (cm *ContextManager) BuildQueryContext(ctx context.Context,
    query string, maxTokens int) ([]*ContextFile, error) {
    cm.mu.RLock()
    defer cm.mu.RUnlock()
    if cm.Massive == nil {
        return nil, fmt.Errorf("massive context not initialized")
    }
    return cm.Massive.LoadContextForQuery(ctx, query, maxTokens)
}

```

Anti-Bluff Test

```

# 1. Clone a large repo (e.g., kubernetes ~20M tokens)
helix init --project /tmp/kubernetes --index

# 2. Verify index built
ls .helix/index/*.json | wc -l # Should have indexed files

# 3. Query for relevant context
helix context "How does the API server handle authentication?"
# Should return < 2M tokens but include relevant files like:
# - pkg/kubeapiserver/authenticator/config.go
# - staging/src/k8s.io/apiserver/pkg/authentication/...

# 4. Verify token count within limit
helix context stats # Should show < 2M loaded

# 5. Verify structural following
# The context should include files connected by imports to the
    top results

```

Integration Verification



Tree-sitter parses Go, JS, TS, Python, Rust correctly



Embedding retriever returns relevant files for queries



Total loaded context never exceeds 2M tokens

Feature 3: Version-Controlled Model Packs

Source Location (in Plandex)

- `app/server/types/model.go` — Model role types (planner, coder, architect, etc.)
- `app/server/hooks/` — Model configuration hooks
- `app/server/model/name.go` — Model name resolution
- Docs: `plandex models`, `plandex set-model`, `plandex model-packs`

Target Location (in HelixCode)

- **NEW:** `internal/llm/model_packs.go` — Model pack definitions and versioning
- **NEW:** `internal/llm/model_pack_store.go` — Git-based storage for model configs
- **MODIFY:** `internal/llm/` — Integrate model packs into provider selection

Exact Code Changes

NEW FILE: `internal/llm/model_packs.go`

```
package llm

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "time"

    "github.com/google/uuid"
)

// ModelRole represents the functional role in a plan.
type ModelRole string

const (
    RolePlanner      ModelRole = "planner"      // High-level
        task planning
    RoleArchitect    ModelRole = "architect"    // System design
    RoleCoder         ModelRole = "coder"        // Implementation
    RoleBuilder       ModelRole = "builder"      // File edits
)
```

```

RoleWholeBuilder ModelRole =
    "wholeFileBuilder" // Whole-file rewrites
RoleSummarizer ModelRole = "summarizer" // Context
    summarization
RoleNamer ModelRole = "names" // Identifier
    naming
RoleCommitMsg ModelRole = "commitMessages"
RoleAutoContinue ModelRole = "autoContinue"
)

// ModelConfig is a single model assignment with settings.
type ModelConfig struct {
    ModelID string `json:"modelId"`
    Temperature float64 `json:"temperature,omitempty"`
    TopP float64 `json:"topP,omitempty"`
    LargeContextFallback string
        `json:"largeContextFallback,omitempty"`
    LargeOutputFallback string
        `json:"largeOutputFallback,omitempty"`
    ErrorFallback string
        `json:"errorFallback,omitempty"`
    StrongModel string `json:"strongModel,omitempty"`
}

// ModelPack is a complete configuration of models per role.
type ModelPack struct {
    ID string `json:"id"`
    Name string `json:"name"`
    Description string `json:"description"`
    LocalProvider string
        `json:"localProvider,omitempty"`
    Roles map[ModelRole]ModelConfig `json:"roles"`
    CreatedAt time.Time `json:"createdAt"`
    UpdatedAt time.Time `json:"updatedAt"`
}

// DefaultModelPack is the Plandex-inspired default multi-model
pack.
func DefaultModelPack() *ModelPack {
    return &ModelPack{
        ID: uuid.New().String(),
        Name: "default",
        Description: "Default multi-model pack optimized for
quality and cost",
        Roles: map[ModelRole]ModelConfig{
            RolePlanner: {
                ModelID: "anthropic/claude-opus-4",
                Temperature: 0.7,
            },
        },
    }
}

```

```

    },
    RoleArchitect: {
        ModelID: "anthropic/claude-sonnet-4",
        Temperature: 0.5,
    },
    RoleCoder: {
        ModelID: "anthropic/claude-sonnet-4",
        Temperature: 0.3,
    },
    RoleBuilder: {
        ModelID: "openai/o3-mini",
        Temperature: 0.2,
        ErrorFallback: "anthropic/claude-sonnet-4",
    },
    RoleWholeBuilder: {
        ModelID: "anthropic/claude-sonnet-4",
        LargeContextFallback: "google/gemini-2.5-pro",
        LargeOutputFallback: "openai/o4-mini-low",
    },
    RoleSummarizer: {
        ModelID: "anthropic/claude-3.5-haiku",
    },
    RoleNamer: {
        ModelID: "anthropic/claude-3.5-haiku",
    },
    RoleCommitMsg: {
        ModelID: "anthropic/claude-3.5-haiku",
    },
    RoleAutoContinue: {
        ModelID: "anthropic/claude-3.5-haiku",
    },
},
CreatedAt: time.Now(),
UpdatedAt: time.Now(),
}
}

```

```

// GetModelForRole returns the effective model for a role.
func (mp *ModelPack) GetModelForRole(role ModelRole)
    (ModelConfig, error) {
    cfg, ok := mp.Roles[role]
    if !ok {
        // Fall back to planner for unconfigured roles
        cfg, ok = mp.Roles[RolePlanner]
        if !ok {
            return ModelConfig{},
                fmt.Errorf("no model configured for role %s", role)
        }
    }
}

```

```

    }
}
return cfg, nil
}

// ResolveModelID applies fallback logic based on context size
// and errors.
func (mp *ModelPack) ResolveModelID(role ModelRole,
contextTokens int, hadError bool) string {
    cfg, _ := mp.GetModelForRole(role)
    if hadError && cfg.ErrorFallback != "" {
        return cfg.ErrorFallback
    }
    if contextTokens > 500_000 && cfg.LargeContextFallback != "" {
        return cfg.LargeContextFallback
    }
    return cfg.ModelID
}

// ModelPackVersion tracks a snapshot of a model pack for
// rollback.
type ModelPackVersion struct {
    Hash      string    `json:"hash"`
    Pack      ModelPack `json:"pack"`
    CommitMsg string    `json:"commitMsg"`
    CommittedAt time.Time `json:"committedAt"`
}

```

NEW FILE: internal/llm/model_pack_store.go

```

package llm

import (
    "encoding/json"
    "fmt"
    "os"
    "path/filepath"
    "sort"
    "time"

    "github.com/go-git/go-git/v5"
    "github.com/go-git/go-git/v5/plumbing/object"
)

// ModelPackStore uses a git repository for version-controlled
// model configurations.
type ModelPackStore struct {

```

```

RepoPath string
repo      *git.Repository
}

func NewModelPackStore(path string) (*ModelPackStore, error) {
    if err := os.MkdirAll(path, 0755); err != nil {
        return nil, err
    }
    repo, err := git.PlainInit(path, false)
    if err != nil {
        // If already exists, open it
        repo, err = git.PlainOpen(path)
        if err != nil {
            return nil, err
        }
    }
    return &ModelPackStore{RepoPath: path, repo: repo}, nil
}

// SavePack writes a model pack and commits it.
func (mps *ModelPackStore) SavePack(pack *ModelPack, message
    string) error {
    pack.UpdatedAt = time.Now()
    data, err := json.MarshalIndent(pack, "", "  ")
    if err != nil {
        return err
    }
    path := filepath.Join(mps.RepoPath, pack.Name+".json")
    if err := os.WriteFile(path, data, 0644); err != nil {
        return err
    }

    wt, err := mps.repo.Worktree()
    if err != nil {
        return err
    }
    _, err = wt.Add(pack.Name + ".json")
    if err != nil {
        return err
    }
    _, err = wt.Commit(message, &git.CommitOptions{
        Author: &object.Signature{
            Name:  "HelixCode",
            Email: "system@helix.code",
            When: time.Now(),
        },
    })
    return err
}

```

```

}

// ListVersions returns all commits for a pack.
func (mps *ModelPackStore) ListVersions(packName string)
    ([]ModelPackVersion, error) {
    ref, err := mps.repo.Head()
    if err != nil {
        return nil, err
    }
    iter, err := mps.repo.Log(&git.LogOptions{From: ref.Hash()})
    if err != nil {
        return nil, err
    }

    var versions []ModelPackVersion
    iter.ForEach(func(c *object.Commit) error {
        // Check if this commit touched our file
        file, err := c.File(packName + ".json")
        if err != nil {
            return nil
        }
        content, err := file.Contents()
        if err != nil {
            return nil
        }
        var pack ModelPack
        if err := json.Unmarshal([]byte(content), &pack); err !=
            nil {
            return nil
        }
        versions = append(versions, ModelPackVersion{
            Hash:      c.Hash.String(),
            Pack:        pack,
            CommitMsg:   c.Message,
            CommittedAt: c.Author.When,
        })
        return nil
    })

    sort.Slice(versions, func(i, j int) bool {
        return
            versions[i].CommittedAt.After(versions[j].CommittedAt)
    })
    return versions, nil
}

```

// CheckoutVersion restores a pack to a specific git commit.


```

func (mps *ModelPackStore) CheckoutVersion(packName, hash
    string) (*ModelPack, error) {
    commit, err := mps.repo.CommitObject(plumbing.NewHash(hash))
    if err != nil {
        return nil, err
    }
    file, err := commit.File(packName + ".json")
    if err != nil {
        return nil, err
    }
    content, err := file.Contents()
    if err != nil {
        return nil, err
    }
    var pack ModelPack
    if err := json.Unmarshal([]byte(content), &pack); err != nil
    {
        return nil, err
    }
    return &pack, nil
}

```

MODIFY: internal/llm/ (integration point)

Add ModelPackResolver interface to the LLM client:

```

type PackAwareClient struct {
    Client *LLMClient
    Pack   *ModelPack
    Store  *ModelPackStore
}

func (pac *PackAwareClient) GenerateForRole(ctx context.Context,
    role ModelRole, prompt string, maxTokens int)
    (*GenerationResult, error) {
    modelID := pac.Pack.ResolveModelID(role, maxTokens, false)
    return pac.Client.Generate(ctx, modelID, prompt, maxTokens)
}

```

Anti-Bluff Test

```

# 1. Create custom model pack
helix model-packs create "fast-pack" --description "Speed
    optimized"
# Set planner=openai/gpt-4.1, builder=openai/o3-mini

# 2. Verify saved to git
helix model-packs log fast-pack # Shows commit history

```

```
# 3. Apply pack to current plan
helix set-model fast-pack

# 4. Run a plan, observe fast models used
helix plan "refactor auth" --auto
# Check logs: builder should use o3-mini, not Claude

# 5. Edit pack (change builder to claude-sonnet)
helix set-model fast-pack builder anthropic/claude-sonnet-4

# 6. Verify new commit
helix model-packs log fast-pack | wc -l # Should be 2

# 7. Checkout old version
helix model-packs checkout fast-pack {hash-v1}
helix model-packs show fast-pack # builder should be o3-mini
again
```

Integration Verification

- ☐ Git repo `.helix/model-packs/` contains versioned JSON files
 - ☐ `set-model` creates a new commit
 - ☐ `checkout` restores previous configuration
 - ☐ Fallback models activate on context size / errors
-

Feature 4: Plan-Driven Execution

Source Location (in Plandex)

- `app/server/model/plan/` — Plan creation, step execution
- `app/server/model/plan/builder.go` — Plan builder
- `app/server/types/active_plan.go` — `ActivePlan` with operations queue
- `app/server/model/prompts/` — Plan prompts (planner, architect, coder)

Target Location (in HelixCode)

- **NEW:** `internal/workflow/plan.go` — Plan domain model
- **NEW:** `internal/workflow/plan_executor.go` — Step-by-step executor
- **NEW:** `internal/workflow/plan_store.go` — Plan persistence
- **MODIFY:** `internal/agent/agent.go` — Integrate plan-driven mode
- **MODIFY:** `cmd/cli/` — `plan`, `step`, `run` commands

Exact Code Changes

NEW FILE: internal/workflow/plan.go

```
package workflow

import (
    "time"

    "github.com/google/uuid"
)

// Plan is a multi-step development task.
type Plan struct {
    ID            string    `json:"id"`
    Name          string    `json:"name"`
    Description    string    `json:"description"`
    Steps         []*PlanStep `json:"steps"`
    Status        PlanStatus `json:"status"`
    Branch        string    `json:"branch"`
    ContextIDs    []string  `json:"context_ids"`
    CreatedAt     time.Time `json:"created_at"`
    UpdatedAt     time.Time `json:"updated_at"`
}

type PlanStatus string
const (
    PlanStatusDraft      PlanStatus = "draft"
    PlanStatusPending    PlanStatus = "pending" // Awaiting
    approval
    PlanStatusRunning    PlanStatus = "running"
    PlanStatusPaused     PlanStatus = "paused"
    PlanStatusCompleted  PlanStatus = "completed"
    PlanStatusFailed     PlanStatus = "failed"
)

// PlanStep is a single unit of work within a plan.
type PlanStep struct {
    ID            string    `json:"id"`
    Index         int       `json:"index"`
    Title         string    `json:"title"`
    Description    string    `json:"description"`
    Status        StepStatus `json:"status"`
    Dependencies   []string  `json:"dependencies"` // Step
    IDs that must complete first
    Files         []string  `json:"files"` // Files to edit
    ContextFiles  []string  `json:"context_files"`
}
```

```

    ModelRole      string          `json:"model_role"`
    Prompt         string          `json:"prompt"`
    Result         *StepResult      `json:"result,omitempty"`
    CreatedAt      time.Time        `json:"created_at"`
    StartedAt      *time.Time       `json:"started_at,omitempty"`
    CompletedAt    *time.Time       `json:"completed_at,omitempty"`
}

```

```

type StepStatus string
const (
    StepStatusPending  StepStatus = "pending"
    StepStatusApproved StepStatus =
        "approved" // User approved before run
    StepStatusRunning  StepStatus = "running"
    StepStatusReview   StepStatus = "review" // Needs hunk
        review
    StepStatusCompleted StepStatus = "completed"
    StepStatusFailed    StepStatus = "failed"
    StepStatusSkipped   StepStatus = "skipped"
)

```

```

type StepResult struct {
    Edits      map[string]string `json:"edits"` // path ->
        new content
    Commands   []string
        `json:"commands"` // Terminal commands to run
    Tests      []string `json:"tests"` // Test
        commands
    Output     string    `json:"output"`
    Error      string    `json:"error,omitempty"`
}

```

// NewPlan creates a plan from a high-level description.

```

func NewPlan(name, description, branch string) *Plan {
    return &Plan{
        ID:          uuid.New().String(),
        Name:        name,
        Description: description,
        Branch:     branch,
        Steps:      []*PlanStep{},
        Status:     PlanStatusDraft,
        CreatedAt:  time.Now(),
        UpdatedAt:  time.Now(),
    }
}

```

// CanExecute checks if a step's dependencies are satisfied.

```

func (p *Plan) CanExecute(stepID string) bool {

```

```

    step := p.GetStep(stepID)
    if step == nil {
        return false
    }
    for _, depID := range step.Dependencies {
        dep := p.GetStep(depID)
        if dep == nil || dep.Status != StepStatusCompleted {
            return false
        }
    }
    return true
}

func (p *Plan) GetStep(id string) *PlanStep {
    for _, s := range p.Steps {
        if s.ID == id {
            return s
        }
    }
    return nil
}

```

NEW FILE: internal/workflow/plan_executor.go

```

package workflow

import (
    "context"
    "fmt"
    "sync"
    "time"

    "dev.helix.code/internal/editor"
    "dev.helix.code/internal/llm"
)

// PlanExecutor runs plan steps with approval gates and
// dependency tracking.
type PlanExecutor struct {
    mu          sync.RWMutex
    Plan        *Plan
    Sandbox     *editor.DiffSandbox
    LLM         *llm.PackAwareClient
    Storage     PlanStore

    stepApprovalCh chan string // Step IDs awaiting approval
    haltOnReview   bool        // Pause between steps for review
}

```

```

func NewPlanExecutor(plan *Plan, sandbox *editor.DiffSandbox,
    llm *llm.PackAwareClient, store PlanStore) *PlanExecutor
{
    return &PlanExecutor{
        Plan:          plan,
        Sandbox:        sandbox,
        LLM:            llm,
        Storage:        store,
        stepApprovalCh: make(chan string),
        haltOnReview:   true,
    }
}

// Run executes the plan step-by-step.
func (pe *PlanExecutor) Run(ctx context.Context) error {
    pe.Plan.Status = PlanStatusRunning
    pe.Storage.Save(pe.Plan)

    for _, step := range pe.Plan.Steps {
        if err := pe.executeStep(ctx, step); err != nil {
            pe.Plan.Status = PlanStatusFailed
            pe.Storage.Save(pe.Plan)
            return fmt.Errorf("step %s failed: %w", step.ID, err)
        }
    }

    pe.Plan.Status = PlanStatusCompleted
    pe.Storage.Save(pe.Plan)
    return nil
}

func (pe *PlanExecutor) executeStep(ctx context.Context, step
    *PlanStep) error {
    if !pe.Plan.CanExecute(step.ID) {
        return fmt.Errorf("dependencies not satisfied for step
            %s", step.ID)
    }

    step.Status = StepStatusPending
    pe.Storage.Save(pe.Plan)

    // Approval gate
    if pe.haltOnReview {
        step.Status = StepStatusPending
        pe.Storage.Save(pe.Plan)
        // Signal UI/CLI that approval is needed
        pe.stepApprovalCh <- step.ID
    }
}

```

```

        // In actual implementation, wait for approval via
        // channel or API
        // <-pe.approvalReceivedCh
    }

    step.Status = StepStatusRunning
    now := time.Now()
    step.StartedAt = &now
    pe.Storage.Save(pe.Plan)

    // Generate edits via LLM for this step
    result, err := pe.generateStepEdits(ctx, step)
    if err != nil {
        step.Status = StepStatusFailed
        step.Result = &StepResult{Error: err.Error()}
        pe.Storage.Save(pe.Plan)
        return err
    }

    // Stage edits in sandbox
    for path, content := range result.Edits {
        original := "" // Load from disk
        pe.Sandbox.StageEdit(path, original, content)
    }

    step.Status = StepStatusReview
    step.Result = result
    pe.Storage.Save(pe.Plan)

    if pe.haltOnReview {
        // Wait for hunk-level review before continuing
        pe.stepApprovalCh <- step.ID + ":review"
    }

    // After review, apply if approved
    if step.Status == StepStatusApproved || !pe.haltOnReview {
        _, err := pe.Sandbox.ApplyAll(".")
        if err != nil {
            return err
        }
        step.Status = StepStatusCompleted
        completed := time.Now()
        step.CompletedAt = &completed
    }

    pe.Storage.Save(pe.Plan)
    return nil
}

```

```

func (pe *PlanExecutor) generateStepEdits(ctx context.Context,
    step *PlanStep) (*StepResult, error) {
    modelID :=
        pe.LLM.Pack.ResolveModelID(llm.ModelRole(step.ModelRole),
            0, false)
    prompt := fmt.Sprintf("Implement the following step:
        \n\n%s\n\nFiles: %v", step.Description, step.Files)
    resp, err := pe.LLM.Client.Generate(ctx, modelID, prompt,
        8000)
    if err != nil {
        return nil, err
    }
    // Parse response into file edits (simplified)
    return &StepResult{
        Edits: map[string]string{step.Files[0]: resp.Text},
        Output: resp.Text,
    }, nil
}

// ApproveStep allows a step to proceed (called by UI/CLI).
func (pe *PlanExecutor) ApproveStep(stepID string) {
    pe.mu.Lock()
    defer pe.mu.Unlock()
    step := pe.Plan.GetStep(stepID)
    if step != nil && step.Status == StepStatusPending {
        step.Status = StepStatusApproved
        pe.Storage.Save(pe.Plan)
    }
}

// RejectStep skips a step.
func (pe *PlanExecutor) RejectStep(stepID string) {
    pe.mu.Lock()
    defer pe.mu.Unlock()
    step := pe.Plan.GetStep(stepID)
    if step != nil {
        step.Status = StepStatusSkipped
        pe.Storage.Save(pe.Plan)
    }
}

```

MODIFY: cmd/cli/ (new commands)

Add Cobra commands:

```

// helix plan create "Refactor auth" --branch feature/auth
// helix plan steps                    # List steps
// helix plan approve {step-id}       # Approve pending step

```



```
// helix plan reject {step-id}      # Reject step
// helix plan run                    # Execute approved plan
```

Anti-Bluff Test

```
# 1. Create a plan
helix plan create "Add JWT middleware"
# Plan has 3 steps generated by planner model:
#   [1] Create JWT config struct
#   [2] Add middleware function
#   [3] Wire into router

# 2. Execute with halt-on-review
helix plan run --review-each-step

# 3. Step 1 runs, halts for review
helix plan status # Shows step-1 as "review"

# 4. Inspect sandbox diff
helix sandbox diff

# 5. Approve step 1
helix plan approve step-1

# 6. Verify step 2 waits for step 1
# Edit step-1 status to "failed" in DB
# Verify step-2 cannot execute

# 7. Complete all steps
helix plan run
# Should execute steps in dependency order
```

Integration Verification

- ☐ Plan steps execute in dependency order
 - ☐ Halt-on-review pauses before applying edits
 - ☐ Sandbox accumulates changes across all steps
 - ☐ Plan state persisted to database
-

Feature 5: Context Building

Source Location (in Plandex)

- `app/server/db/context.go` — Context database model
- `app/server/types/active_plan.go` — `Contexts`, `ContextsByPath`, `AutoLoadContextCh`
- CLI: `plandex load`, `plandex ls` — Manual context loading

Target Location (in HelixCode)

- **NEW**: `internal/context/auto_builder.go` — Automatic context assembly
- **NEW**: `internal/context/relevance.go` — Relevance scoring
- **MODIFY**: `internal/context/context_manager.go` — Integrate auto-context
- **MODIFY**: `internal/discovery/` — Import chain following

Exact Code Changes

NEW FILE: `internal/context/auto_builder.go`

```
package context

import (
    "context"
    "fmt"
    "path/filepath"
    "strings"
)

// AutoBuilder assembles context automatically based on task
// description.
type AutoBuilder struct {
    Indexer *ProjectIndexer
    Massive *MassiveContext
}

func NewAutoBuilder(indexer *ProjectIndexer, massive
    *MassiveContext) *AutoBuilder {
    return &AutoBuilder{Indexer: indexer, Massive: massive}
}

// BuildContext automatically detects relevant files for a task.
func (ab *AutoBuilder) BuildContext(ctx context.Context, task
    string, maxTokens int) ([]*ContextFile, error) {
    // Step 1: Semantic search for seed files
    seeds, err := ab.Massive.LoadContextForQuery(ctx, task,
        maxTokens/4)
    if err != nil {
        return nil, fmt.Errorf("seed search: %w", err)
    }
}
```

```

}

// Step 2: Follow import chains outward
related := ab.expandContext(seeds, maxTokens/2)

// Step 3: Add structural neighbors (same package/directory)
neighbors := ab.addNeighbors(related, maxTokens/4)

// Step 4: Deduplicate and sort by relevance
return ab.deduplicateAndRank(seeds, related, neighbors,
    task, maxTokens), nil
}

```

```

func (ab *AutoBuilder) expandContext(files []*ContextFile,
    tokenBudget int) []*ContextFile {
    var result []*ContextFile
    seen := make(map[string]bool)
    for _, f := range files {
        if seen[f.Path] {
            continue
        }
        seen[f.Path] = true
        result = append(result, f)
        related := ab.Indexer.GetRelatedFiles(f.Path)
        for _, r := range related {
            if seen[r] {
                continue
            }
            seen[r] = true
            cf, _ := ab.Massive.loadFile(r)
            if cf != nil {
                result = append(result, cf)
            }
        }
    }
    return result
}

```

```

func (ab *AutoBuilder) addNeighbors(files []*ContextFile,
    tokenBudget int) []*ContextFile {
    var result []*ContextFile
    seen := make(map[string]bool)
    for _, f := range files {
        dir := filepath.Dir(f.Path)
        for _, other := range ab.Indexer.GetAllFiles() {
            if filepath.Dir(other.Path) == dir && !
                seen[other.Path] {
                seen[other.Path] = true
            }
        }
    }
    return result
}

```

```

        cf, _ := ab.Massive.loadFile(other.Path)
        if cf != nil {
            result = append(result, cf)
        }
    }
}
return result
}

func (ab *AutoBuilder) deduplicateAndRank(seeds, related,
    neighbors []*ContextFile, task string, maxTokens int)
    []*ContextFile {
    all := append(append(seeds, related...), neighbors...)
    seen := make(map[string]bool)
    var unique []*ContextFile
    for _, f := range all {
        if seen[f.Path] {
            continue
        }
        seen[f.Path] = true
        unique = append(unique, f)
    }

    // Score by task relevance (simplified: task words in file
    // content)
    taskWords := strings.Fields(strings.ToLower(task))
    for _, f := range unique {
        score := 0
        content := strings.ToLower(f.Content)
        for _, w := range taskWords {
            if strings.Contains(content, w) {
                score += 10
            }
        }
        // Boost seed files
        for _, s := range seeds {
            if s.Path == f.Path {
                score += 100
            }
        }
        f.Score = score // Add Score field to ContextFile
    }

    // Sort by score descending
    sortContextFilesByScore(unique)

    var result []*ContextFile

```

```

tokensUsed := 0
for _, f := range unique {
    if tokensUsed+f.Tokens > maxTokens {
        break
    }
    result = append(result, f)
    tokensUsed += f.Tokens
}
return result
}

```

Anti-Bluff Test

```

# 1. In a Go project, request: "Add rate limiting to the API"
helix context auto "Add rate limiting to the API"

```

```

# 2. Verify relevant files loaded automatically:
# - api/router.go (semantic match)
# - api/middleware.go (import chain)
# - config/rate_limit.go (same package neighbor)
# - main.go (imports router)

```

```

# 3. Verify token count < 2M
helix context stats # Should show reasonable count

```

```

# 4. Verify no irrelevant files (no test data, no vendor)
helix context list | grep -v "_test.go" | grep -v "vendor/"

```

Integration Verification

☐

Auto-context includes files relevant to the task

☐

Import chains are followed up to 3 levels deep

☐

Token budget respected

Feature 6: Multi-Model Support

Source Location (in Plandex)

- `app/server/model/litellm.go` — LiteLLM proxy integration for 11+ providers
- `app/server/model/client.go` — Model client routing
- `app/server/model/client_stream.go` — Streaming responses
- `app/server/model/model_error.go` — Provider fallback on errors

Target Location (in HelixCode)

- **MODIFY:** internal/llm/ — Add multi-provider routing
- **NEW:** internal/llm/provider_router.go — Provider selection logic
- **NEW:** internal/llm/fallback_chain.go — Error fallback chain
- **MODIFY:** internal/llm/model_packs.go — Already added in Feature 3

Exact Code Changes

NEW FILE: internal/llm/provider_router.go

```
package llm

import (
    "context"
    "fmt"
    "sync"
)

// ProviderRouter selects the best provider for a model ID.
type ProviderRouter struct {
    mu          sync.RWMutex
    providers   map[string]Provider // provider name -> client
    // modelID -> preferred provider
    modelPreferences map[string]string
}

type Provider interface {
    Generate(ctx context.Context, modelID, prompt string,
        maxTokens int) (*GenerationResult, error)
    GenerateStream(ctx context.Context, modelID, prompt string,
        maxTokens int) (<-chan StreamChunk, error)
    EstimateTokens(text string) int
}

type GenerationResult struct {
    Text      string
    TokensIn  int
    TokensOut int
    ModelID   string
    Provider  string
}

type StreamChunk struct {
    Text  string
    Done  bool
    Error error
}
```

```

func NewProviderRouter() *ProviderRouter {
    return &ProviderRouter{
        providers:      make(map[string]Provider),
        modelPreferences: make(map[string]string),
    }
}

func (pr *ProviderRouter) RegisterProvider(name string, p
    Provider) {
    pr.mu.Lock()
    defer pr.mu.Unlock()
    pr.providers[name] = p
}

func (pr *ProviderRouter) ResolveProvider(modelID string)
    (Provider, string, error) {
    pr.mu.RLock()
    defer pr.mu.RUnlock()

    // Parse provider prefix (e.g., "anthropic/claude-sonnet-4")
    parts := strings.SplitN(modelID, "/", 2)
    providerName := parts[0]
    actualModel := modelID
    if len(parts) == 2 {
        actualModel = parts[1]
    }

    provider, ok := pr.providers[providerName]
    if !ok {
        // Fallback to default provider
        if defaultP, ok := pr.providers["default"]; ok {
            return defaultP, actualModel, nil
        }
        return nil, "", fmt.Errorf("no provider for %s", modelID)
    }
    return provider, actualModel, nil
}

func (pr *ProviderRouter) Generate(ctx context.Context, modelID,
    prompt string, maxTokens int) (*GenerationResult, error)
{
    provider, actualModel, err := pr.ResolveProvider(modelID)
    if err != nil {
        return nil, err
    }
    return provider.Generate(ctx, actualModel, prompt, maxTokens)
}

```

NEW FILE: internal/llm/fallback_chain.go

```
package llm

import (
    "context"
    "fmt"
)

// FallbackChain attempts multiple models in sequence on failure.
type FallbackChain struct {
    Router *ProviderRouter
    Pack   *ModelPack
}

func (fc *FallbackChain) GenerateWithFallback(ctx
    context.Context, role ModelRole, prompt string,
    maxTokens int, hadError bool) (*GenerationResult, error)
{
    modelID := fc.Pack.ResolveModelID(role, maxTokens, hadError)
    result, err := fc.Router.Generate(ctx, modelID, prompt,
        maxTokens)
    if err != nil {
        // Try error fallback if configured
        cfg, _ := fc.Pack.GetModelForRole(role)
        if cfg.ErrorFallback != "" {
            return fc.Router.Generate(ctx, cfg.ErrorFallback,
                prompt, maxTokens)
        }
        return nil, fmt.Errorf("primary and fallback failed:
        %w", err)
    }
    return result, nil
}
```

Anti-Bluff Test

```
# 1. Configure multiple providers
helix provider add openrouter --api-key $OPENROUTER_KEY
helix provider add anthropic --api-key $ANTHROPIC_KEY

# 2. Create pack with role-based models
helix model-packs create "multi"
# planner=anthropic/claude-opus-4
# builder=openai/o3-mini
# summarizer=google/gemini-2.5-flash

# 3. Execute plan and verify model routing
helix plan run --verbose-models
```



```
# Logs should show different providers per role

# 4. Test fallback
# Temporarily break anthropic key
export ANTHROPIC_API_KEY=invalid
helix plan run
# Should fall back to error_fallback model (e.g., openai/gpt-4.1)
```

Integration Verification

- ☐ Different models used for different roles
 - ☐ Fallback activates on API errors
 - ☐ Context-size fallback activates >500k tokens
-

Feature 7: Streaming Plan Updates

Source Location (in Plandex)

- `app/server/types/active_plan.go` — `Stream()`, `Subscribe()`, `streamCh`, `streamMessageBuffer`
- `app/server/model/client_stream.go` — Model streaming
- `plandex-shared/types.go` — `StreamMessage` types

Target Location (in HelixCode)

- **NEW**: `internal/workflow/stream.go` — Plan streaming infrastructure
- **NEW**: `internal/server/websocket.go` — WebSocket streaming endpoint
- **MODIFY**: `internal/workflow/plan_executor.go` — Stream updates during execution

Exact Code Changes

NEW FILE: `internal/workflow/stream.go`

```
package workflow

import (
    "encoding/json"
    "sync"
    "time"
)

// StreamMessageType identifies the kind of streaming update.
type StreamMessageType string
```

```

const (
    StreamMessagePlanCreated    StreamMessageType = "plan_created"
    StreamMessageStepStarted    StreamMessageType = "step_started"
    StreamMessageStepThinking    StreamMessageType =
        "step_thinking"
    StreamMessageStepEdit        StreamMessageType = "step_edit"
    StreamMessageStepReview      StreamMessageType = "step_review"
    StreamMessageStepCompleted    StreamMessageType =
        "step_completed"
    StreamMessageDiffHunk        StreamMessageType = "diff_hunk"
    StreamMessageError           StreamMessageType = "error"
    StreamMessageFinished        StreamMessageType = "finished"
)

// StreamMessage is a single streaming update.
type StreamMessage struct {
    Type          StreamMessageType `json:"type"`
    PlanID        string           `json:"plan_id"`
    StepID        string           `json:"step_id,omitempty"`
    HunkID        string           `json:"hunk_id,omitempty"`
    Content       string           `json:"content,omitempty"`
    FilePath      string           `json:"file_path,omitempty"`
    OldText       string           `json:"old_text,omitempty"`
    NewText       string           `json:"new_text,omitempty"`
    Timestamp     time.Time        `json:"timestamp"`
}

// PlanStreamer broadcasts plan updates to subscribers.
type PlanStreamer struct {
    mu          sync.RWMutex
    subscribers map[string]chan StreamMessage
    bufferSize  int
    flushInterval time.Duration
}

func NewPlanStreamer() *PlanStreamer {
    return &PlanStreamer{
        subscribers: make(map[string]chan StreamMessage),
        bufferSize:  100,
        flushInterval: 70 * time.Millisecond,
    }
}

func (ps *PlanStreamer) Subscribe() (string, <-chan
    StreamMessage) {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    id := uuid.New().String()

```

```

    ch := make(chan StreamMessage, ps.bufferSize)
    ps.subscribers[id] = ch
    return id, ch
}

func (ps *PlanStreamer) Unsubscribe(id string) {
    ps.mu.Lock()
    defer ps.mu.Unlock()
    if ch, ok := ps.subscribers[id]; ok {
        close(ch)
        delete(ps.subscribers, id)
    }
}

func (ps *PlanStreamer) Broadcast(msg StreamMessage) {
    ps.mu.RLock()
    defer ps.mu.RUnlock()
    for _, ch := range ps.subscribers {
        select {
        case ch <- msg:
        default: // Drop if buffer full
        }
    }
}

func (ps *PlanStreamer) StreamStepThinking(planID, stepID, chunk
    string) {
    ps.Broadcast(StreamMessage{
        Type:      StreamMessageStepThinking,
        PlanID:    planID,
        StepID:    stepID,
        Content:   chunk,
        Timestamp: time.Now(),
    })
}

func (ps *PlanStreamer) StreamDiffHunk(planID, stepID, hunkID,
    filePath, oldText, newText string) {
    ps.Broadcast(StreamMessage{
        Type:      StreamMessageDiffHunk,
        PlanID:    planID,
        StepID:    stepID,
        HunkID:    hunkID,
        FilePath:  filePath,
        OldText:   oldText,
        NewText:   newText,
        Timestamp: time.Now(),
    })
}

```

```
    })  
}
```

MODIFY: internal/workflow/plan_executor.go

Add streaming to executeStep:

```
func (pe *PlanExecutor) executeStep(ctx context.Context, step  
    *PlanStep) error {  
    // ... existing setup ...  
  
    // Stream thinking tokens as they arrive  
    streamCh, _ := pe.LLM.Client.GenerateStream(ctx, modelID,  
        prompt, 8000)  
    var fullResponse strings.Builder  
    for chunk := range streamCh {  
        if chunk.Error != nil {  
            return chunk.Error  
        }  
        fullResponse.WriteString(chunk.Text)  
        pe.LLM.Streamer.StreamStepThinking(pe.Plan.ID, step.ID,  
            chunk.Text)  
    }  
  
    // Parse and stream diff hunks  
    result := pe.parseResponse(fullResponse.String())  
    for path, edit := range result.Edits {  
        pe.Sandbox.StageEdit(path, original, edit)  
        edit := pe.Sandbox.PendingEdits[path]  
        for _, hunk := range edit.Hunks {  
            pe.LLM.Streamer.StreamDiffHunk(pe.Plan.ID, step.ID,  
                hunk.ID, path, hunk.OldText, hunk.NewText)  
        }  
    }  
  
    // ... rest of step execution ...  
}
```

Anti-Bluff Test

1. Connect WebSocket to plan stream

```
wscat -c ws://localhost:8080/api/v1/plans/{plan-id}/stream
```

2. Start a plan

```
helix plan run
```

3. Verify real-time messages received:

```
# - plan_created
```

```
# - step_started (step-1)
```

```
# - step_thinking (token chunks streaming)
# - step_edit (file path, diff hunks)
# - diff_hunk (hunk-level detail)
# - step_review (step-1 needs approval)
# - step_completed (after approval)
# - step_started (step-2)
# - finished

# 4. Verify live preview: before approval, the diff is visible
# in the WebSocket stream but NOT on disk.
```

Integration Verification

- ☐ WebSocket receives messages in real-time
 - ☐ Thinking tokens stream as they arrive from LLM
 - ☐ Diff hunks stream immediately after generation
 - ☐ Stream buffers correctly (70ms rate limit)
-

Feature 8: Branch Management

Source Location (in Plandex)

- `app/server/db/` — Branch data model
- CLI: `plandex checkout`, `plandex branch` — Branch operations
- Plans are isolated per branch; `plandex apply` affects working tree on that branch

Target Location (in HelixCode)

- **NEW**: `internal/workflow/branch.go` — Plan branch model
- **NEW**: `internal/workflow/branch_manager.go` — Branch lifecycle
- **MODIFY**: `internal/session/session.go` — Track active branch
- **MODIFY**: `cmd/cli/` — `checkout`, `branch` commands

Exact Code Changes

NEW FILE: `internal/workflow/branch.go`

```
package workflow

import (
    "time"

    "github.com/google/uuid"
)
```

```
// PlanBranch isolates a plan's history from other branches.
type PlanBranch struct {
    ID          string `json:"id"`
    Name        string `json:"name"`
    PlanID      string `json:"plan_id"`
    BaseBranch  string `json:"base_branch"` // Git branch this
                    // plan branch is based on
    CurrentState string `json:"current_state"`
    CreatedAt   time.Time `json:"created_at"`
}

func NewPlanBranch(name, planID, baseBranch string) *PlanBranch {
    return &PlanBranch{
        ID:          uuid.New().String(),
        Name:        name,
        PlanID:      planID,
        BaseBranch:  baseBranch,
        CurrentState: "initial",
        CreatedAt:   time.Now(),
    }
}
```

NEW FILE: internal/workflow/branch_manager.go

```
package workflow

import (
    "fmt"
    "os"
    "os/exec"
    "sync"
)

// BranchManager creates and switches plan branches.
type BranchManager struct {
    mu          sync.RWMutex
    branches    map[string]*PlanBranch // branch name -> branch
    active      string
    baseDir     string
}

func NewBranchManager(baseDir string) *BranchManager {
    return &BranchManager{
        branches: make(map[string]*PlanBranch),
        baseDir:  baseDir,
    }
}
```

```

// Checkout creates a new plan branch from the current git
// branch.
func (bm *BranchManager) Checkout(name, planID string)
    (*PlanBranch, error) {
    bm.mu.Lock()
    defer bm.mu.Unlock()

    // Get current git branch
    cmd := exec.Command("git", "-C", bm.baseDir, "branch", "--
        show-current")
    out, err := cmd.Output()
    if err != nil {
        return nil, fmt.Errorf("git branch: %w", err)
    }
    gitBranch := strings.TrimSpace(string(out))

    branch := NewPlanBranch(name, planID, gitBranch)
    bm.branches[name] = branch
    bm.active = name

    // Create git branch if it doesn't exist
    exec.Command("git", "-C", bm.baseDir, "checkout", "-b",
        name).Run()

    return branch, nil
}

// Switch changes the active plan branch.
func (bm *BranchManager) Switch(name string) error {
    bm.mu.Lock()
    defer bm.mu.Unlock()

    if _, ok := bm.branches[name]; !ok {
        return fmt.Errorf("branch %s not found", name)
    }
    bm.active = name
    // Switch git branch
    cmd := exec.Command("git", "-C", bm.baseDir, "checkout",
        name)
    return cmd.Run()
}

// Merge applies changes from one plan branch to another.
func (bm *BranchManager) Merge(from, to string) error {
    bm.mu.Lock()
    defer bm.mu.Unlock()

    // In real implementation, this would merge sandbox states

```

```

    // For now, use git merge as the underlying mechanism
    cmd := exec.Command("git", "-C", bm.baseDir, "merge", from)
    out, err := cmd.CombinedOutput()
    if err != nil {
        return fmt.Errorf("merge failed: %s", out)
    }
    return nil
}

func (bm *BranchManager) Active() string {
    bm.mu.RLock()
    defer bm.mu.RUnlock()
    return bm.active
}

```

Anti-Bluff Test

```

# 1. Create main plan on main branch
helix plan create "Feature A"
helix checkout feature-a

# 2. Create alternative branch for same plan
helix checkout feature-a-alt

# 3. Run different model packs on each branch
helix set-model fast-pack # On feature-a-alt
helix plan run

# 4. Switch back to feature-a
helix checkout feature-a
# Verify plan state is independent (different steps/diffs)

# 5. Apply on feature-a
helix plan apply

# 6. Try to merge feature-a-alt
helix merge feature-a-alt
# Should show conflicts if both modified same files

```

Integration Verification

- ☐ Branches are isolated (different sandbox states)
 - ☐ Git branch created per plan branch
 - ☐ Switching branches updates active sandbox
-

Feature 9: History Management

Source Location (in Plandex)

- `app/server/db/plan.go` — Plan history storage
- CLI: `plandex log`, `plandex rewind` — History viewing and time travel
- `app/server/types/active_plan.go` — Version control for every action

Target Location (in HelixCode)

- **NEW:** `internal/workflow/history.go` — History model
- **NEW:** `internal/workflow/history_store.go` — Persistence
- **MODIFY:** `internal/workflow/plan_executor.go` — Record every action
- **MODIFY:** `cmd/cli/` — `log`, `rewind`, `diff` commands

Exact Code Changes

NEW FILE: `internal/workflow/history.go`

```
package workflow

import (
    "time"

    "github.com/google/uuid"
)

// HistoryEvent records every significant action on a plan.
type HistoryEvent struct {
    ID          string    `json:"id"`
    PlanID      string    `json:"plan_id"`
    Type        EventType `json:"type"`
    StepID      string    `json:"step_id,omitempty"`
    Description string    `json:"description"`
    Before      interface{} `json:"before,omitempty"` // State
                        snapshot before
    After       interface{} `json:"after,omitempty"`  // State
                        snapshot after
    CreatedAt   time.Time  `json:"created_at"`
}

type EventType string
const (
    EventContextAdded      EventType = "context_added"
    EventContextRemoved    EventType = "context_removed"
    EventPromptSent        EventType = "prompt_sent"
    EventResponseRecv      EventType = "response_received"
    EventStepBuilt         EventType = "step_built"
    EventHunkRejected      EventType = "hunk_rejected"
```

```

        EventHunkApplied      EventType = "hunk_applied"
        EventModelChanged     EventType = "model_changed"
        EventPlanRewound      EventType = "plan_rewound"
    )

    // History provides time travel for plans.
    type History struct {
        Events []*HistoryEvent
    }

    func (h *History) Add(event *HistoryEvent) {
        event.ID = uuid.New().String()
        event.CreatedAt = time.Now()
        h.Events = append(h.Events, event)
    }

    func (h *History) GetEvents(planID string) []*HistoryEvent {
        var out []*HistoryEvent
        for _, e := range h.Events {
            if e.PlanID == planID {
                out = append(out, e)
            }
        }
        return out
    }

    // Rewind removes events after a specific event ID.
    func (h *History) Rewind(eventID string) error {
        for i, e := range h.Events {
            if e.ID == eventID {
                h.Events = h.Events[:i+1]
                return nil
            }
        }
        return fmt.Errorf("event %s not found", eventID)
    }

```

MODIFY: internal/workflow/plan_executor.go

Add history recording hooks:

```

func (pe *PlanExecutor) executeStep(ctx context.Context, step
    *PlanStep) error {
    // Record step start
    pe.recordEvent(EventStepBuilt, step.ID, "Step execution
        started", nil, step)

    // ... execute ...

```

```

    // Record step completion
    pe.recordEvent(EventStepBuilt, step.ID, "Step completed",
        step, result)
    return nil
}

func (pe *PlanExecutor) recordEvent(t EventType, stepID, desc
    string, before, after interface{}) {
    pe.Storage.RecordHistory(&HistoryEvent{
        PlanID:      pe.Plan.ID,
        Type:         t,
        StepID:       stepID,
        Description:  desc,
        Before:       before,
        After:        after,
    })
}

```

Anti-Bluff Test

1. Run a multi-step plan

helix plan run --auto

2. View history

helix log

Output:

[1] context_added: loaded main.go

[2] prompt_sent: "Add JWT auth"

[3] step_built: step-1 "Create JWT struct"

[4] hunk_applied: main.go:1

[5] step_built: step-2 "Add middleware"

3. Rewind to before step-2

helix rewind 3 # Rewind to event [3]

4. Verify step-2 removed from plan

helix plan status # Should show only step-1

5. Verify file reverted

main.go should not have middleware code

6. Compare versions

helix diff 4 5 # Show difference between event 4 and 5

Integration Verification

☐

Every plan action creates a history event

☐

- ☐ `rewind` restores plan to previous state
 - ☐ `diff` between versions shows changes
 - ☐ File system changes are reverted on `rewind`
-

Feature 10: Collaboration Features

Source Location (in Plandex)

- `app/server/db/` — Multi-user org/plan data model
- `app/server/handlers/` — Multi-user API endpoints
- `app/server/notify/` — Real-time notifications
- Plans can be shared; multiple users can contribute

Target Location (in HelixCode)

- **NEW:** `internal/session/collaboration.go` — Multi-user session model
- **NEW:** `internal/server/plan_collab.go` — Collaboration API
- **MODIFY:** `internal/workflow/plan.go` — Ownership and permissions
- **MODIFY:** `internal/notification/` — Conflict notifications

Exact Code Changes

NEW FILE: `internal/session/collaboration.go`

```
package session

import (
    "sync"
    "time"

    "github.com/google/uuid"
)

// SharedPlan allows multiple users to collaborate on a plan.
type SharedPlan struct {
    mu sync.RWMutex

    PlanID      string          `json:"plan_id"`
    OwnerID     string          `json:"owner_id"`
    OrgID       string          `json:"org_id"`
    Participants map[string]*Participant `json:"participants"`
    Locks       map[string]*FileLock   `json:"locks" //
                filePath -> lock
    CreatedAt   time.Time          `json:"created_at"`
}
```

```

type Participant struct {
    UserID    string    `json:"user_id"`
    Role      string    `json:"role" // "owner", "editor",
                "viewer"
    JoinedAt  time.Time `json:"joined_at"`
    LastSeen  time.Time `json:"last_seen"`
}

type FileLock struct {
    UserID    string    `json:"user_id"`
    FilePath  string    `json:"file_path"`
    AcquiredAt time.Time `json:"acquired_at"`
}

func NewSharedPlan(planID, ownerID, orgID string) *SharedPlan {
    return &SharedPlan{
        PlanID:      planID,
        OwnerID:     ownerID,
        OrgID:       orgID,
        Participants: make(map[string]*Participant),
        Locks:        make(map[string]*FileLock),
        CreatedAt:    time.Now(),
    }
}

func (sp *SharedPlan) AddParticipant(userID, role string) {
    sp.mu.Lock()
    defer sp.mu.Unlock()
    sp.Participants[userID] = &Participant{
        UserID:    userID,
        Role:      role,
        JoinedAt:  time.Now(),
        LastSeen:  time.Now(),
    }
}

func (sp *SharedPlan) LockFile(userID, filePath string) bool {
    sp.mu.Lock()
    defer sp.mu.Unlock()
    if _, locked := sp.Lock[filePath]; locked {
        return false
    }
    sp.Lock[filePath] = &FileLock{
        UserID:    userID,
        FilePath:   filePath,
        AcquiredAt: time.Now(),
    }
    return true
}

```

```

}

func (sp *SharedPlan) UnlockFile(userID, filePath string) {
    sp.mu.Lock()
    defer sp.mu.Unlock()
    if lock, ok := sp.Locks[filePath]; ok && lock.UserID ==
        userID {
        delete(sp.Locks, filePath)
    }
}

func (sp *SharedPlan) HasConflict(filePath string) bool {
    sp.mu.RLock()
    defer sp.mu.RUnlock()
    _, locked := sp.Locks[filePath]
    return locked
}

```

NEW FILE: internal/server/plan_collab.go

```

package server

import (
    "encoding/json"
    "net/http"

    "dev.helix.code/internal/session"
    "dev.helix.code/internal/workflow"
)

func (s *Server) handleJoinPlan(w http.ResponseWriter, r
    *http.Request) {
    planID := r.URL.Query().Get("plan_id")
    userID := r.Context().Value("user_id").(string)

    shared, err := s.CollabStore.Get(planID)
    if err != nil {
        http.Error(w, "plan not found", http.StatusNotFound)
        return
    }
    shared.AddParticipant(userID, "editor")
    s.CollabStore.Save(shared)

    json.NewEncoder(w).Encode(shared)
}

func (s *Server) handleLockFile(w http.ResponseWriter, r
    *http.Request) {

```

```

var req struct {
    PlanID    string `json:"plan_id"`
    FilePath  string `json:"file_path"`
}
json.NewDecoder(r.Body).Decode(&req)
userID := r.Context().Value("user_id").(string)

shared, _ := s.CollabStore.Get(req.PlanID)
if shared.LockFile(userID, req.FilePath) {
    w.WriteHeader(http.StatusOK)
} else {
    http.Error(w, "file locked by another user",
        http.StatusConflict)
}
}

```

Anti-Bluff Test

```

# 1. User A creates a shared plan
helix plan create "Shared Feature" --org my-org
helix share --plan shared-feature --user user-b

```

```

# 2. User B joins
helix join shared-feature # As user-b

```

```

# 3. User A locks a file
helix lock shared-feature --file api/auth.go

```

```

# 4. User B tries to edit same file
# Should get 409 Conflict or warning

```

```

# 5. Both users stage edits on different files
# User A edits api/auth.go
# User B edits api/user.go
# Both should succeed

```

```

# 6. Apply all changes
# Plan should merge both users' sandbox changes

```

Integration Verification

☐

Multiple users can join a shared plan

☐

File locks prevent concurrent edits

☐

Conflict notifications sent to participants

☐

Plan history shows who made each change

Summary of New/Modified Files

New Files (21 files)

1. `internal/editor/diff_sandbox.go`
2. `internal/editor/diff_sandbox_test.go`
3. `internal/context/massive_context.go`
4. `internal/context/indexer.go`
5. `internal/context/chunker.go`
6. `internal/context/retriever.go`
7. `internal/context/auto_builder.go`
8. `internal/llm/model_packs.go`
9. `internal/llm/model_pack_store.go`
10. `internal/llm/provider_router.go`
11. `internal/llm/fallback_chain.go`
12. `internal/workflow/plan.go`
13. `internal/workflow/plan_executor.go`
14. `internal/workflow/plan_store.go` (implied from interface usage)
15. `internal/workflow/stream.go`
16. `internal/workflow/branch.go`
17. `internal/workflow/branch_manager.go`
18. `internal/workflow/history.go`
19. `internal/workflow/history_store.go`
20. `internal/session/collaboration.go`
21. `internal/server/plan_collab.go`

Modified Files (8 files)

1. `internal/editor/editor.go` — Integrate sandbox
2. `internal/session/session.go` — SandboxID, branch tracking
3. `internal/context/context_manager.go` — Massive context integration
4. `internal/llm/` — Pack-aware client
5. `internal/agent/agent.go` — Plan-driven mode
6. `cmd/cli/` — New CLI commands
7. `internal/workflow/plan_executor.go` — History + streaming hooks
8. `internal/server/websocket.go` — Streaming endpoint

CLI Commands Added

- `helix review` — Review sandbox diffs
- `helix apply` — Apply accepted sandbox edits
- `helix reject` — Reject sandbox edits
- `helix context auto <query>` — Automatic context assembly
- `helix plan create <name>` — Create a new plan
- `helix plan run` — Execute plan steps
- `helix plan approve <step-id>` — Approve a step
- `helix plan reject <step-id>` — Reject a step
- `helix model-packs create <name>` — Create model pack
- `helix model-packs log <name>` — View pack history
- `helix model-packs checkout <name> <hash>` — Restore pack version
- `helix checkout <branch>` — Create/switch plan branch

- `helix merge <branch>` — Merge plan branch
- `helix log` — View plan history
- `helix rewind <n>` — Rewind plan
- `helix diff <a> <b>` — Compare versions
- `helix share --plan <id> --user <id>` — Share plan
- `helix lock --plan <id> --file <path>` — Lock file for editing

Integration Architecture

HelixCode with Plandex Features

cmd/cli/	← New commands added
internal/	
agent/	
agent.go	← Plan-driven execution mode
editor/	
editor.go	← Integrates DiffSandbox
diff_sandbox.go (NEW)	← Cumulative diff review
diff_editor.go	← Existing (unchanged)
context/	
context_manager.go	← MassiveContext integration
massive_context.go (NEW)	← 2M/20M token handling
indexer.go (NEW)	← Tree-sitter project map
chunker.go (NEW)	← File chunking
retriever.go (NEW)	← Embedding search
auto_builder.go (NEW)	← Automatic context assembly
llm/	
model_packs.go (NEW)	← Versioned model packs
model_pack_store.go (NEW)	← Git-based pack versioning
provider_router.go (NEW)	← Multi-provider routing
fallback_chain.go (NEW)	← Error fallback chain
... existing providers	
workflow/	
plan.go (NEW)	← Plan domain model
plan_executor.go (NEW)	← Step-by-step executor
stream.go (NEW)	← Streaming updates
branch.go (NEW)	← Plan branches
branch_manager.go (NEW)	← Branch lifecycle
history.go (NEW)	← Plan history
history_store.go (NEW)	← History persistence
session/	
session.go	← SandboxID, branch tracking
collaboration.go (NEW)	← Multi-user plans
server/	
websocket.go	← Streaming endpoint
plan_collab.go (NEW)	← Collaboration API
api/	← OpenAPI spec updates

Anti-Bluff End-to-End Master Test

```
#!/bin/bash
set -e

echo "=== MASTER PORTING VERIFICATION ==="

# 0. Setup
cd /tmp/helix-test
rm -rf .helix
helix init --project .

# 1. DIFF SANDBOX (Feature 1)
echo "1. Testing Diff Sandbox..."
echo 'func Hello() string { return "old" }' > hello.go
helix sandbox stage hello.go --edit
    'func Hello() string { return "new" }'
[ "$(cat hello.go)" = 'func Hello() string { return "old" }' ]
    && echo " PASS: Sandbox isolated"
helix sandbox apply
[ "$(cat hello.go)" = 'func Hello() string { return "new" }' ]
    && echo " PASS: Apply works"
helix sandbox revert
[ "$(cat hello.go)" = 'func Hello() string { return "old" }' ]
    && echo " PASS: Revert works"

# 2. MASSIVE CONTEXT (Feature 2)
echo "2. Testing Massive Context..."
helix context auto "How does routing work?"
TOKENS=$(helix context stats --json | jq '.total_tokens')
[ "$TOKENS" -lt 2000000 ] && echo " PASS: Context < 2M tokens"
[ -f .helix/index/project_map.json ] && echo
    " PASS: Tree-sitter index exists"

# 3. MODEL PACKS (Feature 3)
echo "3. Testing Model Packs..."
helix model-packs create "test-pack" --planner openai/gpt-4.1
helix model-packs log test-pack | grep -q "Created" && echo "
    PASS: Pack versioned"
helix set-model test-pack
# Verify via API: should use gpt-4.1 for planner role

# 4. PLAN EXECUTION (Feature 4)
echo "4. Testing Plan Execution..."
helix plan create "Test Plan" --steps 3
helix plan run --review-each-step
# Verify steps execute sequentially, halt for review
```

```

# 5. CONTEXT BUILDING (Feature 5)
echo "5. Testing Context Building..."
helix context auto "Add middleware"
helix context list | grep -q "middleware\|router" && echo "
    PASS: Relevant files loaded"

# 6. MULTI-MODEL (Feature 6)
echo "6. Testing Multi-Model..."
helix model-packs create "multi" --planner anthropic/claude-
    opus-4 --builder openai/o3-mini
# Verify logs show different models per role

# 7. STREAMING (Feature 7)
echo "7. Testing Streaming..."
# WebSocket receives messages in real-time during plan execution

# 8. BRANCH MANAGEMENT (Feature 8)
echo "8. Testing Branches..."
helix checkout test-branch
helix checkout main
# Verify independent plan states

# 9. HISTORY (Feature 9)
echo "9. Testing History..."
helix log | grep -q "step_built" && echo "    PASS: History
    recorded"
helix rewind 1
# Verify state rewound

# 10. COLLABORATION (Feature 10)
echo "10. Testing Collaboration..."
helix share --plan test-plan --user test-user-2
# Verify participant added

echo "=== ALL TESTS PASSED ==="

```

Dependency Additions

Go modules to add to go.mod:

```

github.com/smacker/go-tree-sitter v0.0.0-20240723051150-0a0434df323a
github.com/go-git/go-git/v5 v5.12.0
github.com/google/uuid v1.6.0

```

Submodule additions (via SSH as per HelixCode conventions):

```
# Vector DB for embedding retrieval (if not already present)
git submodule add git@github.com:HelixDevelopment/vectordb.git
    internal/vectordb

# Tree-sitter grammars (lazy loaded)
git submodule add git@github.com:tree-sitter/tree-sitter-go.git
    vendor/tree-sitter-go
```

Risk Assessment & Mitigation

Risk	Impact	Mitigation
Tree-sitter C bindings	Build complexity	Use pure-Go grammar parsers where available; ship precompiled .so for Docker
2M token context memory	OOM on small workers	Lazy chunk loading; streaming tokenization; worker memory limits
Git repo for model packs	.helix/ bloat	Shallow clones; pack pruning; pack compaction
Multi-user conflicts	Data loss	Pessimistic file locking; conflict markers; manual merge UI
WebSocket streaming	Connection drops	Buffered replay; automatic reconnection; event ID sequencing

Rollback Strategy

Each feature is designed to be **toggleable** via feature flags:

```
// internal/config/features.go
type FeatureFlags struct {
    DiffSandbox      bool `env:"HELIX_FEATURE_DIFF_SANDBOX"
        default:"true"`
    MassiveContext   bool `env:"HELIX_FEATURE_MASSIVE_CONTEXT"
        default:"true"`
    ModelPacks       bool `env:"HELIX_FEATURE_MODEL_PACKS"
        default:"true"`
    PlanExecution    bool `env:"HELIX_FEATURE_PLAN_EXECUTION"
        default:"true"`
    Streaming        bool `env:"HELIX_FEATURE_STREAMING"
        default:"true"`
    Collaboration    bool `env:"HELIX_FEATURE_COLLABORATION"
        default:"false"`
}
```

If any feature causes issues, set the env var to `false` and the system falls back to the original HelixCode behavior.

End of Plandex -> HelixCode Porting Plan